
Integrative Programming and Technologies III

Test 1 (Inheritance, Abstract classes and Interfaces)

30 March 2023

Duration: 1 Hour 15

Total: 40 marks

Question 1-10: 10 Marks

Indicate whether the following statements are **TRUE** or **FALSE**.

1. In Java, a class can extend ONLY one superclass. T
2. It is a good programming practice to declare the data members in a superclass in inheritance as public. F
3. Method overload occurs between a superclass and subclass in inheritance. F
4. Method override occurs in the same class. F
5. You can create an object of an abstract class. F
6. You can create an object of an interface. F
7. In polymorphism, the method which is called depends on the type of object. T
8. In polymorphism, an appropriate method of an object is determined when the program is running (i.e., at runtime). T
9. A class must be declared abstract if it contains an abstract method. T
10. An abstract class can implement an interface. T

Question 11 2 Marks

The two main relationships in object-oriented programming are _____ and _____.

Answer: Inheritance✓ and composition✓

Question 12 2 marks

Explain why the **highlighted** code is invalid.

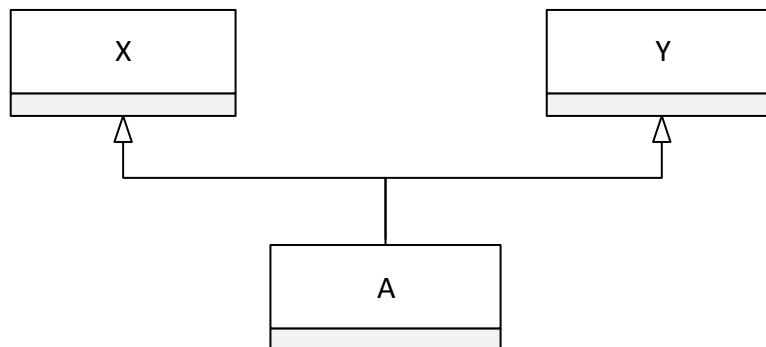
```
public interface IPriceCalculator {  
    public abstract double calcTotalPrice();  
    public abstract int calcTotalPrice();  
}
```

Answer: the code breaks the rule for method overload: both methods have the same parameter list✓✓

Question 13

3 marks

The diagram below shows the classes **A**, **X** and **Y**. If **X** contains only constants and abstract methods **Y** contains variables and method implementations, explain (**do not re-draw** the diagram), what can be done to avoid multiple inheritance.



Answer: 1. Convert one class (X because it contains constants and abstract methods) ✓✓ and 2. Let A inherit Y and implement X ✓

Question 14

5 marks

Class **MyClass** implements an interface **IMyInterface**. State whether the following statements are valid or invalid. Explain your answer.

- | | | |
|------|--|-----------|
| i. | <code>MyClass c=new MyClass();</code> | (1 marks) |
| ii. | <code>IMyInterface i1=new IMyInterface();</code> | (2 marks) |
| iii. | <code>IMyInterface i2=new MyClass();</code> | (2 marks) |

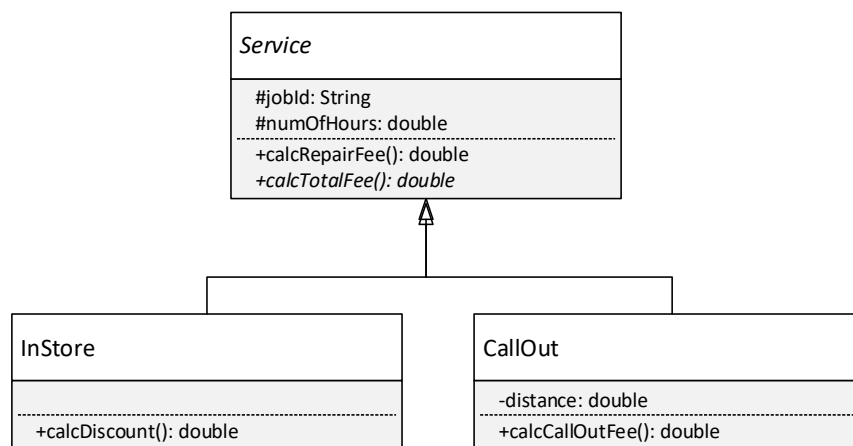
Answer:

- | | | |
|------|--|---|
| i. | <code>MyClass c=new MyClass();</code> | Valid; can instantiate a class ✓ |
| ii. | <code>IMyInterface im=new IMyInterface();</code> | Invalid; cannot instantiate an interface ✓✓ |
| iii. | <code>IMyInterface i=new MyClass();</code> | Valid can assign an object to an interface variable ✓✓ |

Question 15

8 marks

XYZ Locksmith offers two types of services: **in-store** (customers go to the shop with their locks for repair) and **call-out** (the locksmith travels to clients to repair their locks).



The diagram above shows the relationship among the classes **Service**, **InStore** and **CallOut**.

1. Complete the class header of **Service**.

2 marks

```

_____Service {
}
    
```

Answer:

```

public abstract class ✓✓Service {
}
    
```

2. Write the method declarations for **calcRepairFee()** and **calcTotalFee()** in Service.

```

2.1 _____ 2 marks
2.2 _____ 2 marks
    
```

Answer:

```

2.1 public abstract double calcRepairFee(); ✓✓
2.2 public abstract double calcTotalFee(); ✓✓
    
```

3. Complete the class header for class **CallOut**

2 marks

```

_____ CallOut _____ {
}
    
```

Answer:

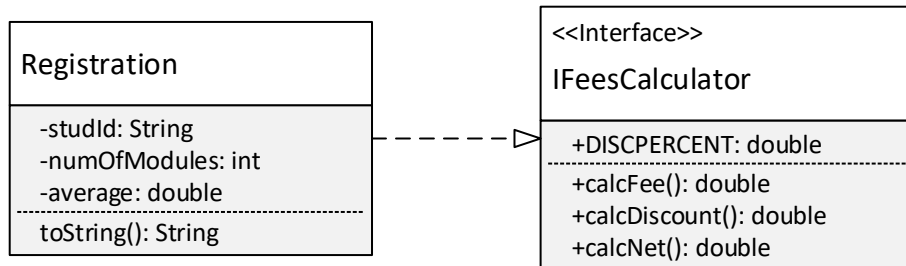
```

public class CallOut extends Service ✓✓{
}
    
```

Question 16

10 marks

NLM University offers discount on fees to students that obtained an average of 75% or more. The diagram below shows the class **Registration** and interface **IFeesCalculator**. Registration contains a student's registration details: **studId** (student id), **numOfModules** (the number of modules registered) and **average** (the average mark). **IFeesCalculator** contains **DISCPERCENT**: the percentage discount (a constant, 25%); **calcFees()**: returns the fees, based on the number of modules registered (**each module is R12 000**); **calcDiscount()**: returns the discount (calculated as 20% of the fees); and **calcNet()**: returns the net fees (calculated as the fees less discount)

1. Complete the code for the interface **IFeesCalculator**

5 marks

```

public interface IFeesCalculator {
    _____
    _____
    _____
    _____
}
  
```

Answer:

```

public interface IFeesCalculator {
    public static final double DISCPERCENT=0.20; ✓✓
    public double calcFee(); ✓
    public double calcDiscount(); ✓
    public double calcNet(); ✓
}
  
```

2. Complete the class header for **Registration**

2 marks

```

_____Registration _____{
}
  
```

Answer

```

public class Registration implements IFeesCalculator ✓✓{
}
  
```

3. Complete the method **calcDiscount()** in **Registration** by calling the **calcFee()**. Assume **calcFee()** has already been implemented. 3 marks

```
public double calcDiscount() {
```

```
    _____  
    _____  
    _____
```

```
}
```

Answer: any correct code ✓✓✓

```
public double calcDiscount() {
```

```
    double ans=0;
```

```
    if(average>=0.75) {✓
```

```
        ans=DISCPERCENT * calcFee();✓
```

```
    }
```

```
    return ans; ✓
```

```
}
```